

1. 배경 및 문제 정의 (Why)

조직 내에서 10대 이상의 서버가 보안이 엄중한 Airgap 환경 하에 있으며, 이 서버들에서 수 많은 Docker 컨테이너들을 운용하고 있습니다. 그러나 각 서버의 Docker 상태를 파악하고 제어하는 과정에서 다음과 같은 한계와 실무적 Pain Point 가 있었습니다.

1. 기존의 상용 관제 솔루션(Portainer 등) 은 서버 규모와 솔루션 기능에 비해서 라이선스 비용 과도
2. 상용 및 무료 오픈소스 솔루션 모두 외부로 무언가 전송하려고 하는 Telemetry 의 위험성 존재
3. 수기 CLI 명령 조작 시 컨테이너·데이터·볼륨 오삭제 리스크
4. 분산 서버 모니터링 및 관리 공수 과다

2. 구현 방법 및 활용 Tool (How / What)

이전에 분산된 nVidia GPU 서버들의 GPU 이용률 현황을 종합적으로 보여주는 nVidia 의 GPU 관제 솔루션, "NVIDIA DCGM - NVIDIA Data Center GPU Manager" 시스템을 사내에 구축해 본 경험이 있으며, 이 경험이 "분산 시스템의 컨테이너 관제 시스템" 을 만드는데 큰 밑천이 되었습니다. 개별 서버의 Agent 와 중앙의 통합 관제하는 Host 시스템 구조를 배웠습니다.

또한 상용 솔루션 [Portainer](#) 는 2대 까지는 무료로 사용해 볼 수 있기 때문에, 자택의 개인 PC에서 따로 hyper-V 를 이용하여 가상머신 시스템을 3대 구축하여 운용경험을 쌓았습니다. 이는 이후 "더티룸-클린룸 기법"으로 이어졌습니다.



그림. PC 1대로 실제 운용 환경과 유사한 다중 서버 환경을 모사하여 상용 솔루션 운용 경험 축적과 자체 개발 솔루션의 개발 및 테스트를 진행할 수 있었던 Windows의 VM 솔루션, Hyper-V 의 화면.

※ AI 클린룸 기법

원하는 프로그램의 기능만 모방하고 원본 소스코드의 저작권 침해를 회피하는 소프트웨어 공학 기법의 하나로 더티룸-클린룸 기법이 있습니다. 이 기법은 원본 코드가 새 코드에 섞여 들어가는 것을 원천 차단하기 위해 철저히 두 단계로 분리하여 진행됩니다.

1. 더티 룸(Dirty Room) - 분석 및 명세화: 첫 번째 그룹(사람 또는 AI)이 목표하는 유료 프로그램의 동작 방식, 입출력 결과, API, 매뉴얼 등을 분석합니다. 이때 내부 소스코드는 보지 않은 채, 겉으로 드러난 기능만을 바탕으로 '기능 명세서(Specification)'를 작성합니다.
2. 클린 룸(Clean Room) - 독립적 개발: 원본 프로그램의 코드를 한 번도 본 적이 없는 완전히 독립된 두 번째 그룹(다른 개발팀이나 다른 독립된 AI 모델)에게 앞서 만든 기능 명세서만 전달합니다. 이들은 명세서의 요구사항만을 보고 밑바닥부터 코드를 새로 작성합니다.

AI를 활용할 때는 특정 챗봇(예: 챗GPT)에게 원본 프로그램을 분석해 기획서를 쓰게 하고, 완전히 다른 챗봇(예: 클로드)에게 그 기획서만 던져주어 코드를 짜게 만드는 식으로 이 과정을 자동화 할 수 있습니다.

구조가 비슷할 GPU 관제 솔루션 구축 경험, 그리고 거의 동일할 상용 솔루션 사용 경험을 합쳐서 "무엇을 만들어야 하는가" 를 열심히 문서화 했습니다.

Host 가 갖춰야 하는 조건, Agent 들이 갖춰야 하는 조건을 각각 Host.md, Agent.md 로 구축한 후, 전체 프로젝트가 갖춰야 하는 조건을 프로젝트 개괄을 설명하는 README.md 파일에 "프로젝트 마일스톤" 을 작성하여 Host 컨테이너와 Agent 컨테이너의 요소들을 빌드하도록 지시하였습니다.

즉, README.md 에 전체 작업 내역이 있으며, 이 작업 중 Host 빌드 시에는 다시 Host.md 참조, 다음 작업으로 Agent를 빌드하게 할 때는 Agent.md 를 참조하는 구조 입니다. 이렇게 명령을 내린 이유는, Host 만 만들라고 하면 Host 의 개성이 생기고, Agent 만 만들라 하면 Agent 의 개성이 생겨서 둘 사이 통신을 하는데 있어서 동일한 형식을 만들지 못할까 우려하였기 때문으로, Host 와 Agent 를 다른 순서로 생성하되, 그 둘을 만드는 맥락을 동일하게 가져가서 둘 사이 연결에 문제가 없는 구조를 보장하기 위함입니다.

README.md, Host.md, Agent.md 문서가 제 손으로 각각 완료되고 난 뒤, Google AI Pro 계정을 연결한 agy(antigravity cli, 클로드 코드 같은 google 의 cli 코딩 도구)를 Gemini 3. 6 Flash 를 Effort: High- Deepest reasoning 으로 세팅한 뒤, 제가 만든 문서를 읽고 평가하라 시켰으며, 여기서 나온 문제점들을 반영하여 이 세개의 구조 문서를 고도화 하였습니다.

```
관리자: C:\WINDOWS\system32\conhost.exe - agy
> /model
  Exited /model command

Switch Model

> Gemini 3.6 Flash (current)
  Gemini 3.5 Flash
  Gemini 3.1 Pro
  Claude Sonnet 4.6 (Thinking)
  Claude Opus 4.6 (Thinking)
  GPT-OSS 120B (Medium)

Effort  [ ] ●-----●-----[ ] [ ]
         low      medium    high
        Deepest reasoning for complex problems - slower but strongest

Keyboard: ↑/↓ Navigate ←/→ Effort enter Select esc Go Back

Gemini 3.6 Flash · hig
```

그림. 계획을 세울 때는 가장 강력한 사고력을 가지도록 Effort: high 모드로 설정하였다.

```
관리자: C:\WINDOWS\system32\conhost.exe - agy
> /model
  Exited /model command

Switch Model

> Gemini 3.6 Flash (current)
  Gemini 3.5 Flash
  Gemini 3.1 Pro
  Claude Sonnet 4.6 (Thinking)
  Claude Opus 4.6 (Thinking)
  GPT-OSS 120B (Medium)

Effort  [ ] ●-----[ ]-----○ [ ]
         low      medium    high
        Balanced speed and reasoning quality for most tasks

Keyboard: ↑/↓ Navigate ←/→ Effort enter Select esc Go Back

Gemini 3.6 Flash · hig
```

그림. 코딩을 할 때는 빠른 반응속도와 사용량 제약 회피를 위해 Effort: medium 으로 설정하였다.

이후 코딩 과정에서는 오히려 Gemini 3.6 Flash 를 Effort: Medium 으로 낮추고, 주어진 문서대로 빌드하라 지시하였습니다.

이 부분은 약간 논쟁의 여지가 있는 부분입니다. "계획을 짜는 부분에서 가장 강력한 사고력을 지닌 모델을 사용한다" 라는 것에는 모두가 동의하지만, 실제 코딩을 하는 부분에서 낮은 모델을 사용하면 아쉽지 않은가 하는 문제제기가 있을 수 있는데, 주어진 일일 사용량 제한과 더 빠른 AI의 응답으로 중간과정을 사람이 실제로 실행해 가면서 휴먼 피드백, 휴먼 인 더 루프 개발을 진행한다는 관점에서 합리화 할 수 있습니다.

agy 에게 전달된 Host, Agent 컨테이너에 관한 문서가, 컨테이너 빌드까지 완성하라는 지시였기 때문

에, 중간중간 생성한 컨테이너 이미지를 각각 VM 들에 전달하여 docker compose 를 사용하여 컨테이너 up, down 을 적절히 수행한 뒤 테스트 하였습니다.

이후 테스트 과정에서 발견한 오류, 기능추가 등등을 각기 적절한 READEM, Host, Agent md 파일에 추가하고 재 빌드 시켰으며, 어느정도 완성되었다 판단한 후, 동료 3명에게 공개하여 동료평가를 진행, 해당 과정에서 받은 피드백을 반영하여 프로젝트를 완료하였습니다.

완성된 프로젝트는 본인의 깃헙, [링크](#) 에서 확인하실 수 있습니다.

Slide 3. System Architecture (전체 시스템 아키텍처)

분산 아키텍처 & 통신 흐름

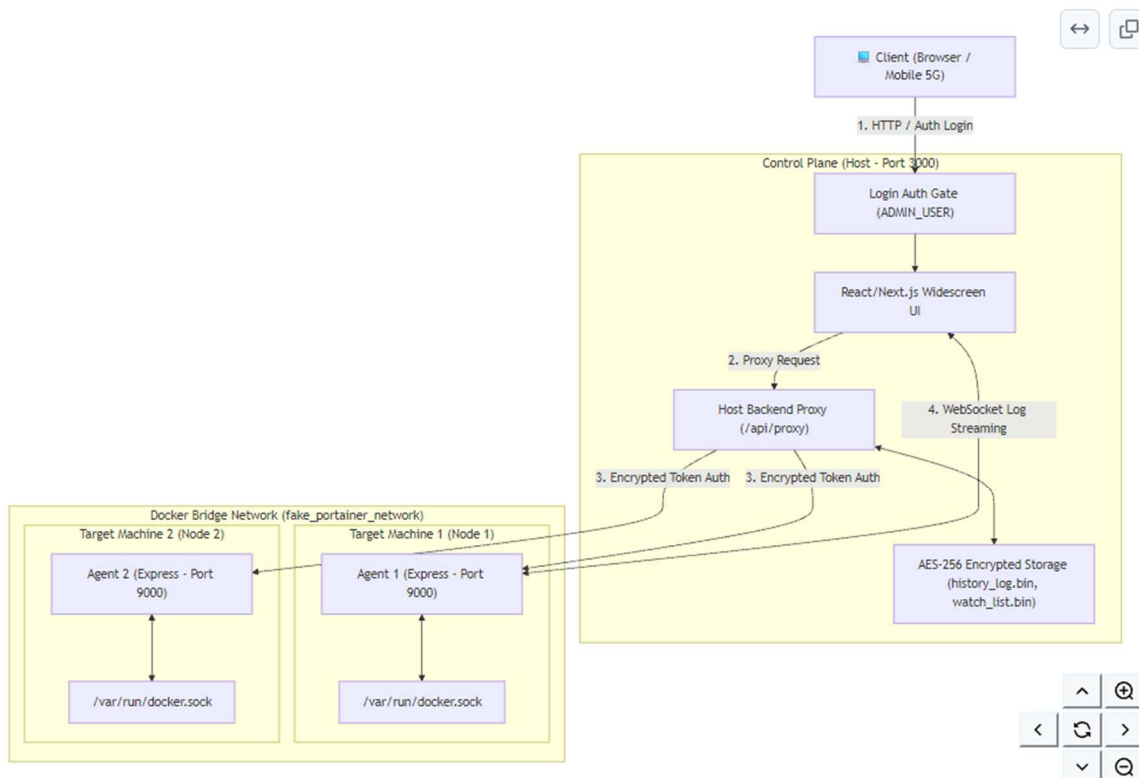


그림. 프로젝트 전체 아키텍처 도식

3. 핵심 가치 (Value Proposition)

3-1. Biz. Impact

본 사례를 통해 달성한 성과를 기술합니다. 기존 방식 대비 개선 정도를 정량 지표(시간/비용 절감, 생산성 향상 등)로 제시합니다.

1. 상용 솔루션 구매를 위한 지출 없이 요구된 기능상 동등하며 보안적으로 더 우수한 솔루션을 라이선스 위반의 위험 없이 자체 개발, 확보하였음.
2. 기존의 수기를 통한 컨테이너 관리

3-2. 확장 가능성

다른 멤버사나 조직에서도 본 사례를 쉽게 이해하고 활용할 수 있도록, 적용 시 필요한 전제 조건과 실행 Know-how를 기술합니다.

3-3. 혁신성

문제 해결 과정에서 기존과 다르게 접근한 차별화 포인트와 새롭게 시도한 방식을 서술합니다.